Project 4: Deep FRAME Texture Synthesis

Shuyang Hou
Yuanpei College
Peking University
2200017797@stu.pku.edu.cn

1 Introduction

Texture modeling is an important problem in computer vision, with applications in graphics, inpainting, and understanding mid-level statistics of natural images. The FRAME (Filters, Random field, And Maximum Entropy) model defines an energy-based distribution over images using the responses of linear filters. *Deep FRAME* extends this idea by using a hierarchical CNN descriptor as the filter bank, allowing the model to capture increasingly complex statistics.

In this lab, I follow the formulation given in the handout (Section 2 & 3) to implement a hierarchical FRAME model:

- A CNN descriptor maps an input image to a feature map at the top layer.
- The total energy (scoring function) is defined as the sum of the top-layer feature responses.
- This energy defines an unnormalized log-density; combined with a Gaussian reference model, it yields a probability distribution over images.
- The model is trained by maximum likelihood, whose gradient is approximated by contrasting the descriptor responses between a target image and synthesized images.
- Synthesized images are updated by Langevin dynamics that samples from the current model.

2 Hierarchical FRAME Model

2.1 FRAME distribution

Let $I \in^{3\times H\times W}$ denote a color image (RGB). The deep FRAME model defines a probability distribution

$$p(I; w) \propto \exp\big(f(I; w)\big)\, q(I), \tag{1}$$

where w denotes the parameters of the CNN descriptor, $f(I; w)$ is the scoring function, and $q(I)$ is a Gaussian reference model (white noise prior).

In our implementation, the reference model is taken as

$$q(I) \propto \exp\left(-\frac{1}{2\sigma^2}\|I\|_2^2\right), \tag{2}$$

which encourages the pixels to stay within a reasonable range and acts as a regularizer on the image.

2.2 Descriptor network

The descriptor network is a small CNN composed of up to three convolutional layers with ReLU activations. Let $F_k^{(l)}$ denote the k-th filter at layer l, and let $[F_k^{(L)} * I](x)$ denote the response of the k-th top-layer filter at spatial location x. The scoring function is defined as

$$f(I; w) = \sum_k \sum_x [F_k^{(L)} * I](x), \tag{3}$$

i.e., the sum over all channels and spatial positions of the final feature map.

In code, this is implemented as:

- Forward pass: $h = \text{Descriptor}(I)$, where h is the top-layer feature map.
- Scoring function: $f(I; w) = \texttt{h.sum()}$.

I consider three depth configurations:

(1) **1-layer model**: one convolutional layer with 3 input channels and 128 output channels, kernel size 15×15, stride 1, padding 7, followed by ReLU.
(2) **2-layer model**: the above plus a second convolutional layer with 128 input channels, 64 output channels, kernel size 5×5, stride 1, padding 2, followed by ReLU.
(3) **3-layer model**: the above plus a third convolutional layer with 64 input channels, 32 output channels, kernel size 3×3, stride 1, padding 1, followed by ReLU.

Table 1 summarizes the architecture of the 3-layer descriptor. For the 1-layer and 2-layer models, we simply truncate at the desired depth.

Layer	Input channels	Output channels	Kernel size	Activation
conv1	3	128	15×15	ReLU
conv2	128	64	5×5	ReLU
conv3	64	32	3×3	ReLU

Table 1: Descriptor architecture for the 3-layer deep FRAME model. The 1- and 2-layer models use only the first 1 or 2 convolutional layers respectively.

2.3 Maximum likelihood learning

Given a target texture image I_{tgt}, the goal is to learn w by maximizing its log-likelihood under $p(I; w)$. The log-likelihood gradient can be written in the general form

$$\frac{\partial L}{\partial w} = \mathbb{E}_{p_{\text{data}}}\left[\frac{\partial f(I; w)}{\partial w}\right] - \mathbb{E}_{p(I;w)}\left[\frac{\partial f(I; w)}{\partial w}\right], \tag{4}$$

where p_{data} is the empirical distribution of the training images (in our case, a single texture image).

In practice, we approximate the expectations by:

- $\mathbb{E}_{p_{\text{data}}}$ using the single target image I_{tgt},
- $\mathbb{E}_{p(I;w)}$ using a synthesized image I_{syn} sampled from the current model by Langevin dynamics.

This leads to the following approximation of the gradient:

$$\frac{\partial L}{\partial w} \approx \frac{\partial f(I_{\text{tgt}}; w)}{\partial w} - \frac{\partial f(I_{\text{syn}}; w)}{\partial w}. \tag{5}$$

We define the objective

$$f_{\text{diff}} = f(I_{\text{tgt}}; w) - f(I_{\text{syn}}; w), \tag{6}$$

and perform gradient ascent on w using

$$w \leftarrow w + \eta \, \frac{\partial f_{\text{diff}}}{\partial w}, \tag{7}$$

where η is the learning rate. In code, this is implemented as:

```
feat_tgt = model(img_tgt)
f_tgt = feat_tgt.sum()

feat_syn = model(img_syn.detach())
f_syn = feat_syn.sum()

f_diff = f_tgt - f_syn
model.zero_grad()
f_diff.backward()
```

2.4 Langevin dynamics sampling

To draw samples from $p(I; w)$, we use Langevin dynamics, which updates the image by following the gradient of the log-density plus Gaussian noise. Using the decomposition

$$\log p(I; w) = f(I; w) - \frac{1}{2\sigma^2}\|I\|_2^2 + \text{const}, \tag{8}$$

the gradient with respect to I is

$$\frac{\partial \log p(I; w)}{\partial I} = \frac{\partial f(I; w)}{\partial I} - \frac{I}{\sigma^2}. \tag{9}$$

The discrete-time Langevin update used in this lab is

$$I_{t+1} = I_t + \frac{\epsilon^2}{2}\left(\frac{\partial f(I_t; w)}{\partial I} - \frac{I_t}{\sigma^2}\right) + \epsilon Z_t, \qquad Z_t \sim \mathcal{N}(0, I), \tag{10}$$

where ϵ is the step size.

In code, this is implemented in the `langevin()` function:

```
x = Variable(x.data, requires_grad=True)
feat = descriptor(x)
f = feat.sum()
f.backward()
grad_x = x.grad
noise = torch.randn_like(x)

x.data += (eps * eps / 2.0) * (grad_x - x / (sigma * sigma)) + eps * noise
```

3 Experimental Setup

3.1 Data and preprocessing

Each image is resized to 224×224 and converted to a tensor with values in $[0, 1]$. For training, we rescale the tensor to $[0, 255]$ and subtract the per-channel mean:

$$I' = 255 \cdot I - \mu, \quad \mu_c = \frac{1}{HW}\sum_x I_c(x), \tag{11}$$

so that the descriptor operates on zero-centered images.

3.2 Implementation details

The implementation is written in PyTorch. The main script `deep_frame.py` trains and samples the deep FRAME model given a chosen depth and texture tag. Important hyper-parameters (as used in the final experiments) are:

- Number of epochs: 2000.
- Langevin step size: $\epsilon = 1.0$.
- Langevin steps per epoch: 10.
- Reference standard deviation: $\sigma = 1.0$.
- Learning rates: a small constant for each layer (e.g., 1×10^{-5}).

All experiments are run either on GPU (if available) or CPU otherwise. A helper script `run_all_experiments.py` is used to run all textures $\times$ all depths in parallel on a multi-GPU machine.

3.3 Model designs

The assignment encourages designing our own structures for the 1-layer and 2-layer models, beyond the provided example. In our experiments, we explore variations of:

- the number of output channels in the first two convolutional layers;
- the kernel sizes in the 1-layer and 2-layer models;
- the Langevin step size and number of steps.

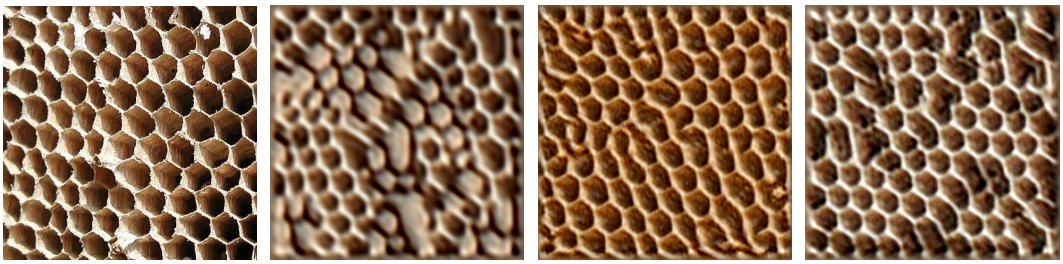
The final "best design" for the beehive texture (used in Section 4.1) is chosen by visually inspecting the sharpness and realism of synthesized textures as well as the stability of training (oscillations of f_{diff}).

4 Results and Analysis

In this section we present the main results required by the assignment: synthesized textures for the beehive image with different depths, and the visualization of the first-layer filters for different images and designs.

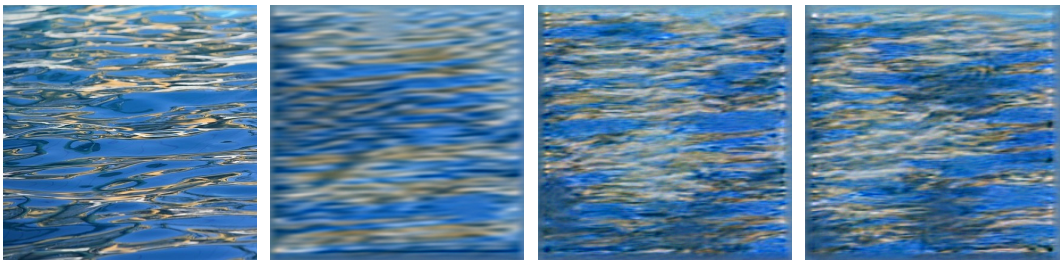
4.1 Synthesized beehive textures with 1, 2, and 3 layers

Figure 1-6 show the synthesized textures for original images using our best 1-layer, 2-layer, and 3-layer designs, along with the target image for reference.



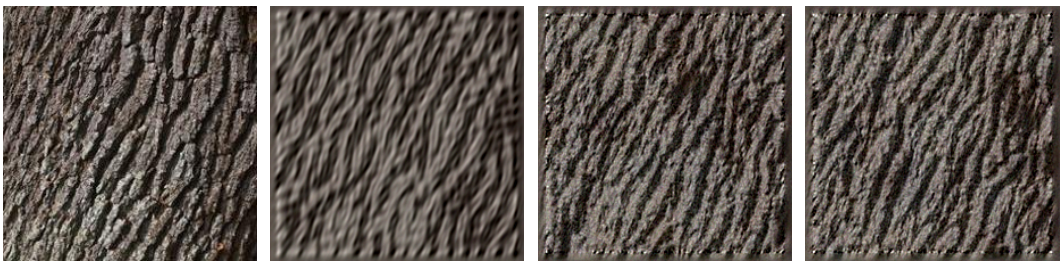
(a) Target (b) 1-layer (c) 2-layer (d) 3-layer

Figure 1: Synthesized beehive textures for our best designs with 1, 2, and 3 layers. (a) Target image; (b)–(d) synthesized samples after training.



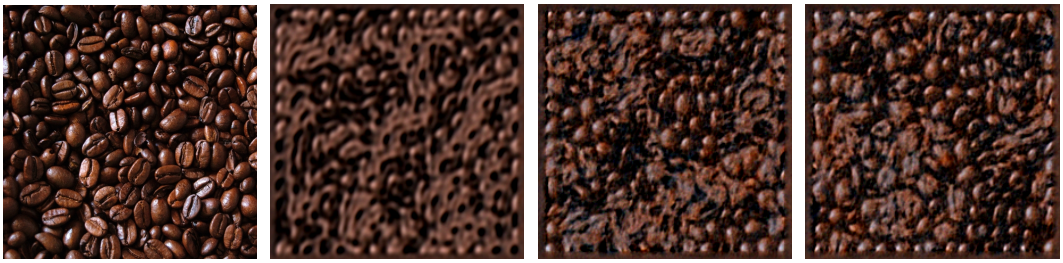
(a) Target (b) 1-layer (c) 2-layer (d) 3-layer

Figure 2: Synthesized water textures for our best designs with 1, 2, and 3 layers. (a) Target image; (b)–(d) synthesized samples after training.



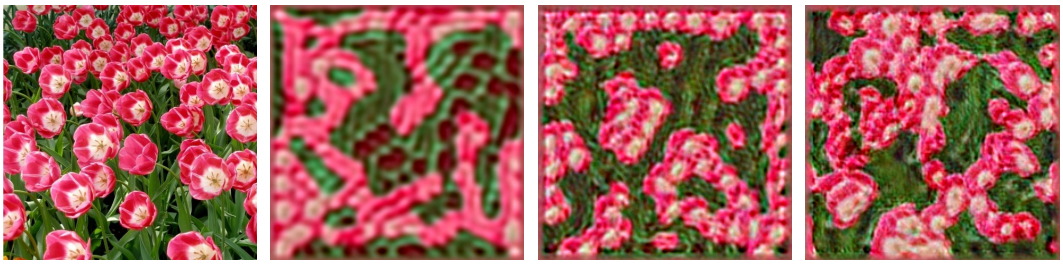
(a) Target (b) 1-layer (c) 2-layer (d) 3-layer

Figure 3: Synthesized bark textures for our best designs with 1, 2, and 3 layers. (a) Target image; (b)–(d) synthesized samples after training.



(a) Target (b) 1-layer (c) 2-layer (d) 3-layer

Figure 4: Synthesized coffee textures for our best designs with 1, 2, and 3 layers. (a) Target image; (b)–(d) synthesized samples after training.



(a) Target (b) 1-layer (c) 2-layer (d) 3-layer

Figure 5: Synthesized rose textures for our best designs with 1, 2, and 3 layers. (a) Target image; (b)–(d) synthesized samples after training.

(a) Target (b) 1-layer (c) 2-layer (d) 3-layer

Figure 6: Synthesized stucco textures for our best designs with 1, 2, and 3 layers. (a) Target image; (b)–(d) synthesized samples after training.

Qualitative comparison. Empirically, the 1-layer model captures only relatively low-level statistics such as local color distributions and coarse edge orientations. The synthesized texture tends to look noisy and may not reproduce the clear hexagonal structure of the beehive.

The 2-layer model improves the global arrangement of patterns: the cells become more regular and the spacing between them is more consistent. However, some fine details along the cell boundaries can still be blurry or broken.

The 3-layer model typically produces the most visually plausible textures. The hexagonal cells appear sharper and more coherent, and larger-scale regularity is better preserved. This suggests that deeper descriptors can better capture higher-order statistics and long-range dependencies important for this texture.

Effect of hyper-parameters. We also observe that the Langevin step size and the number of steps per epoch strongly influence the sharpness of the synthesized images. Too large a step size can lead to unstable images with artifacts, while too small a step size results in slow mixing and blur. The learning rate similarly affects the stability of f_{diff}, which often oscillates rather than increasing monotonically due to the alternating updates of the synthesized image and the descriptor parameters.

4.2 First-layer filter visualization across images

The assignment asks us to visualize the filters in the first layer and compare them across images. In our implementation, we visualize the first-layer filters of the trained descriptor (conv1) as a grid, similar to standard CNN filter visualizations.

Figure 7 shows the conv1 filters learned on several different textures using the 3-layer model.

Observations across images. The filters adapt to the dominant structures in each texture:

- For bark and stucco, many filters resemble oriented edge or curve detectors, reflecting the irregular but locally structured patterns.
- For beehive, filters often pick up hexagonal edges and high-contrast boundaries between cells.
- For water, more wavy and low-frequency patterns appear, capturing the ripple-like structures.

Overall, the first-layer filters behave similarly to Gabor-like or edge-detection filters, but their exact shapes depend on the underlying texture statistics.

4.3 Comparing filters across designs

The assignment further suggests comparing learned filters for different designs (e.g., different depths or different 1-/2-layer architectures).

Figure 8 illustrates an example of such a comparison for the beehive texture: conv1 filters from the 1-layer, 2-layer, and 3-layer models using our best hyper-parameters for each design.

Depth comparison. We observe that:

- In the 1-layer model, the filters are responsible for capturing both local edges and somewhat higher-level patterns; some filters appear noisy or mixed.
- In the 2-layer model, the first-layer filters become slightly more localized and oriented, while the second layer composes them into more complex motifs.
- In the 3-layer model, the first-layer filters often look "cleaner" and simpler (edge or blob detectors), while higher layers capture more complex shapes.

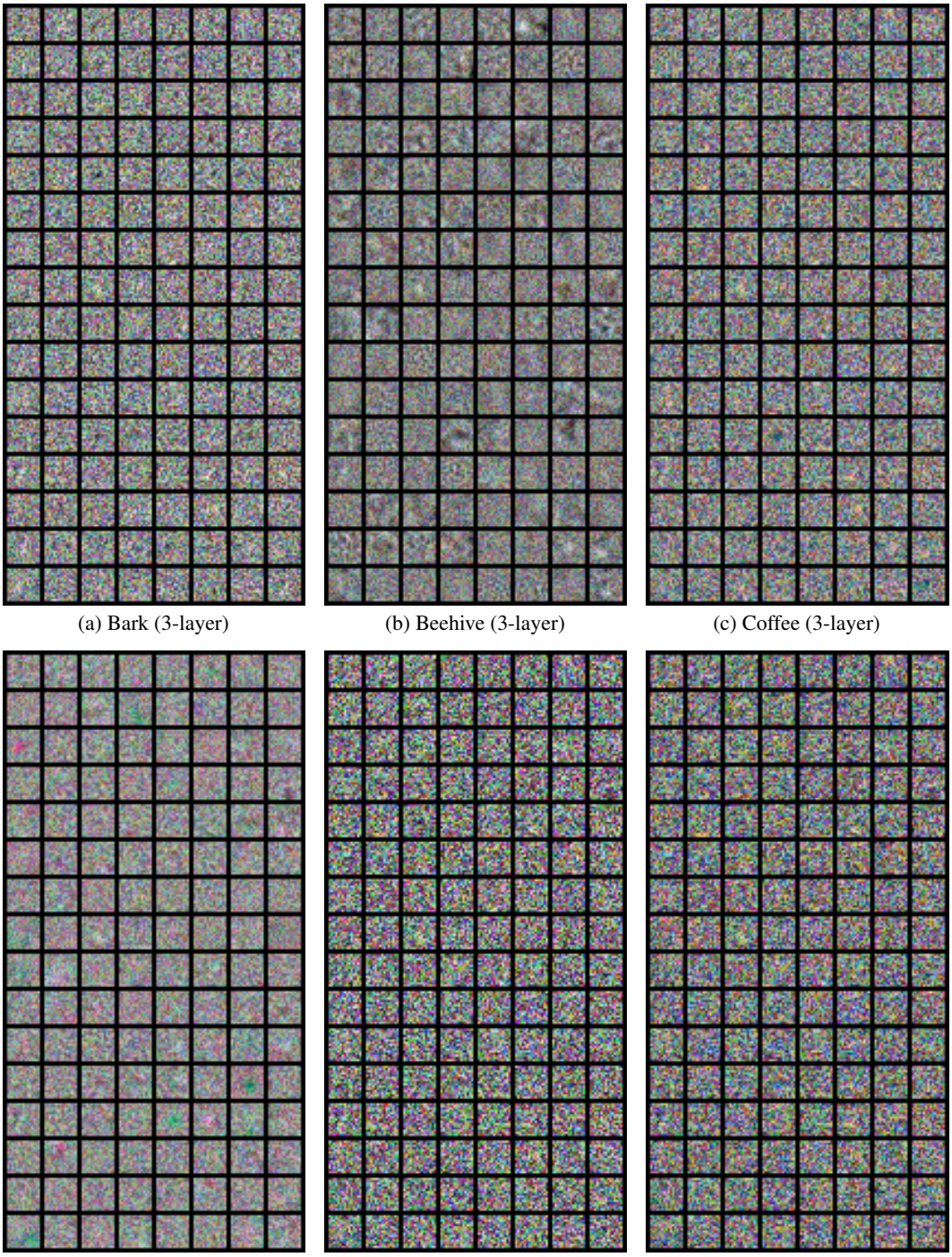
This is consistent with the intuition from deep CNNs: early layers learn generic low-level features, while deeper layers specialize in more abstract patterns. Within the FRAME formulation, these filters are tuned specifically to reproduce the statistics of the texture image under the constraints of the model.

5 Conclusion

In this lab, I implemented a hierarchical FRAME model using a CNN descriptor and studied its behavior for texture synthesis. The model defines a probability distribution over images via a scoring function given by the top-layer feature responses, combined with a Gaussian reference model. Parameters are learned by maximum likelihood, approximated using synthesized images obtained from Langevin dynamics.

The experiments on several textures show that:

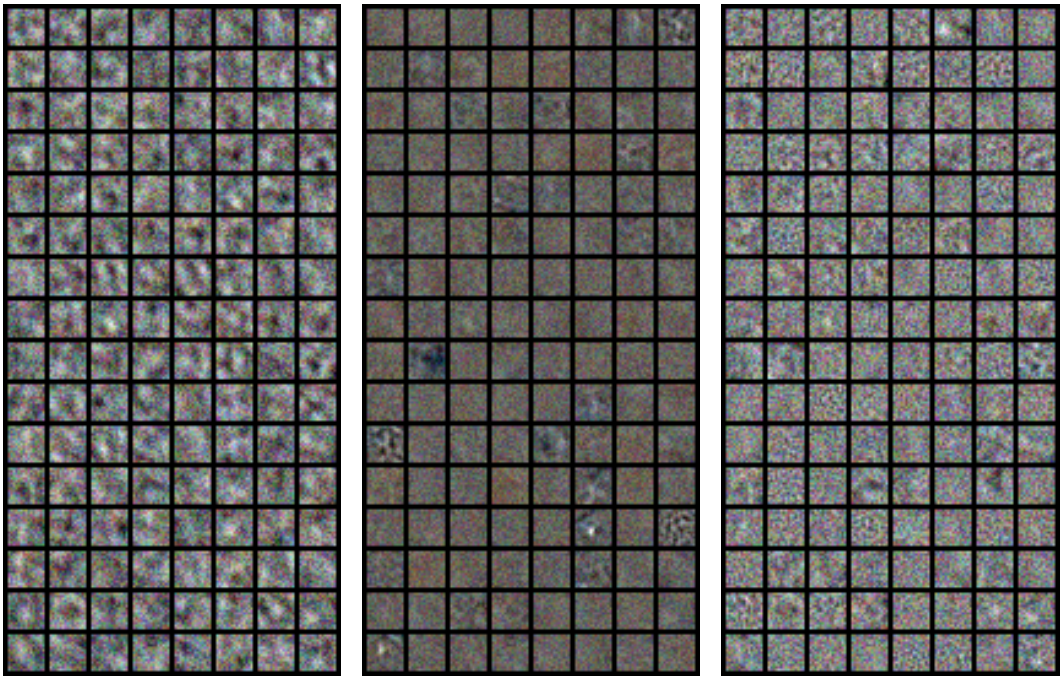
- The deep FRAME model can generate realistic textures that resemble the target images.
- Increasing the depth of the descriptor generally improves the quality of synthesized textures, especially for structured patterns such as the beehive.
- First-layer filters adapt to the underlying texture statistics and resemble edge or blob detectors whose exact shapes depend on the image content.



(a) Bark (3-layer) (b) Beehive (3-layer) (c) Coffee (3-layer)

(d) Rose (3-layer) (e) Stucco (3-layer) (f) Water (3-layer)

Figure 7: Visualization of first-layer filters (conv1) for different textures using the 3-layer model. Each panel shows the learned filters arranged in a grid.



(a) Beehive, 1-layer (b) Beehive, 2-layer (c) Beehive, 3-layer

Figure 8: First-layer filters for the beehive texture under different model designs.